

Tutorial-4

```

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from tqdm import tqdm

# Step 1: Define the neural network architecture
class MLPWithDropout(nn.Module):
    def __init__(self, input_size, hidden_size1, hidden_size2, output_size, dropout_prob):
        super(MLPWithDropout, self).__init__()
        self.fc1 = nn.Linear(input_size, hidden_size1)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(hidden_size1, hidden_size2)
        self.relu2 = nn.ReLU()
        self.fc3 = nn.Linear(hidden_size2, output_size)
        self.dropout = nn.Dropout(dropout_prob)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu1(x)
        x = self.fc2(x)
        x = self.relu2(x)
        x = self.fc3(x)
        x = self.dropout(x)
        return x

# Step 2: Load the CIFAR-10 dataset
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

train_dataset = datasets.CIFAR10(root='./data', train=True, download=True, transform=transform)
test_dataset = datasets.CIFAR10(root='./data', train=False, download=True, transform=transform)

# Step 3: Create DataLoader for training and testing
batch_size = 64

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

# Step 4: Instantiate the model, loss function, and optimizer
input_size = 3 * 32 * 32 # CIFAR-10 images are 32x32x3
hidden_size1 = 256
hidden_size2 = 128
output_size = 10 # Number of classes in CIFAR-10
dropout_probs = [0.1, 0.5] # Different dropout probabilities to experiment with
criterion = nn.CrossEntropyLoss()

# Step 5: Training the model
num_epochs = 10
for dropout_prob in dropout_probs:
    model = MLPWithDropout(input_size, hidden_size1, hidden_size2, output_size, dropout_prob)
    optimizer = optim.Adam(model.parameters(), lr=0.001)

    print(f"----- DROPOUT PROBABILITY: {dropout_prob} -----")

    for epoch in range(num_epochs):
        model.train()
        running_loss = 0.0
        total_train = 0
        correct_train = 0

        for inputs, labels in tqdm(train_loader, desc=f'Epoch {epoch + 1}/{num_epochs}'):
            inputs = inputs.view(inputs.size(0), -1)
            optimizer.zero_grad()

            outputs = model(inputs)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

```

```

running_loss += loss.item()

_, predicted = torch.max(outputs.data, 1)
total_train += labels.size(0)
correct_train += (predicted == labels).sum().item()

train_accuracy = correct_train / total_train
print(f'Training Loss: {running_loss / len(train_loader)}, Training Accuracy: {train_accuracy * 100:.2f}%')

# Step 6: Testing the model
model.eval()
correct = 0
total = 0

with torch.no_grad():
    for inputs, labels in tqdm(test_loader, desc='Testing'):
        inputs = inputs.view(inputs.size(0), -1)
        outputs = model(inputs)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

accuracy = correct / total
print(f'Test Accuracy: {accuracy * 100:.2f}%')

```



Files already downloaded and verified

Files already downloaded and verified

```

----- DROPOUT PROBABILITY: 0.1 -----
Epoch 1/10: 100%|██████████| 782/782 [00:24<00:00, 31.30it/s]
Training Loss: 1.703954680648911, Training Accuracy: 39.67%
Epoch 2/10: 100%|██████████| 782/782 [00:26<00:00, 30.03it/s]
Training Loss: 1.5192631242220358, Training Accuracy: 46.36%
Epoch 3/10: 100%|██████████| 782/782 [00:25<00:00, 30.70it/s]
Training Loss: 1.4211430817918704, Training Accuracy: 50.29%
Epoch 4/10: 100%|██████████| 782/782 [00:25<00:00, 30.59it/s]
Training Loss: 1.358061014767498, Training Accuracy: 52.46%
Epoch 5/10: 100%|██████████| 782/782 [00:25<00:00, 30.39it/s]
Training Loss: 1.291247633793165, Training Accuracy: 54.67%
Epoch 6/10: 100%|██████████| 782/782 [00:25<00:00, 30.54it/s]
Training Loss: 1.2311178894756396, Training Accuracy: 56.69%
Epoch 7/10: 100%|██████████| 782/782 [00:25<00:00, 30.56it/s]
Training Loss: 1.1760631610670358, Training Accuracy: 58.55%
Epoch 8/10: 100%|██████████| 782/782 [00:24<00:00, 31.52it/s]
Training Loss: 1.127231972754154, Training Accuracy: 59.97%
Epoch 9/10: 100%|██████████| 782/782 [00:25<00:00, 30.92it/s]
Training Loss: 1.0837493810964667, Training Accuracy: 61.46%
Epoch 10/10: 100%|██████████| 782/782 [00:25<00:00, 30.56it/s]
Training Loss: 1.0325436855063719, Training Accuracy: 63.17%
Testing: 100%|██████████| 157/157 [00:03<00:00, 41.14it/s]
Test Accuracy: 51.45%
----- DROPOUT PROBABILITY: 0.5 -----
Epoch 1/10: 100%|██████████| 782/782 [00:26<00:00, 29.49it/s]
Training Loss: 1.9912241188156636, Training Accuracy: 28.69%
Epoch 2/10: 100%|██████████| 782/782 [00:26<00:00, 29.49it/s]
Training Loss: 1.8938560234311292, Training Accuracy: 31.83%
Epoch 3/10: 100%|██████████| 782/782 [00:26<00:00, 30.07it/s]
Training Loss: 1.8463273693228621, Training Accuracy: 33.42%
Epoch 4/10: 100%|██████████| 782/782 [00:25<00:00, 30.19it/s]
Training Loss: 1.8102299307313416, Training Accuracy: 34.86%
Epoch 5/10: 100%|██████████| 782/782 [00:25<00:00, 30.24it/s]
Training Loss: 1.7758071754899476, Training Accuracy: 36.07%
Epoch 6/10: 100%|██████████| 782/782 [00:26<00:00, 29.72it/s]
Training Loss: 1.7512875743534253, Training Accuracy: 36.52%
Epoch 7/10: 100%|██████████| 782/782 [00:25<00:00, 30.15it/s]
Training Loss: 1.7263835769175264, Training Accuracy: 37.50%
Epoch 8/10: 100%|██████████| 782/782 [00:26<00:00, 30.05it/s]
Training Loss: 1.7078745234043091, Training Accuracy: 37.67%
Epoch 9/10: 100%|██████████| 782/782 [00:27<00:00, 28.43it/s]
Training Loss: 1.6857742235788604, Training Accuracy: 38.45%
Epoch 10/10: 100%|██████████| 782/782 [00:25<00:00, 30.44it/s]
Training Loss: 1.6656214282335833, Training Accuracy: 38.89%
Testing: 100%|██████████| 157/157 [00:03<00:00, 40.91it/s]Test Accuracy: 52.11%

```